



UT1. MANEJO DE FICHEROS

ACTIVIDAD 3

Título: Escritura y lectura de archivos en Java

Descripción:

En esta actividad vamos a simular el registro de eventos en un **fichero de log de seguridad**. El objetivo es practicar cómo **escribir** y **leer** datos en un archivo de texto con Java, utilizando flujos de salida y entrada.

El programa creará un archivo de log en el que se almacenan algunos mensajes de seguridad ficticios, y después abrirá ese mismo archivo para **mostrar su contenido por consola**.

Objetivos:

- Comprender cómo **escribir texto en un archivo** utilizando la clase **FileWriter**.
- Comprender cómo **leer el contenido de un archivo** utilizando la clase **FileReader**.
- Aprender a leer un fichero línea a línea combinando **FileReader** con **BufferedReader** y el método **readLine()**.
- Familiarizarse con la manipulación de **flujos de datos en Java** aplicados a la gestión de logs.

Instrucciones:

1. Crea una nueva clase llamada **LogActividad3** dentro del proyecto.
2. En el método **main()**, escribe el código para realizar las siguientes operaciones:
 - Crea un archivo llamado **seguridad_actividad3.log** dentro del directorio del proyecto.
 - Escribe en el archivo las siguientes líneas de texto utilizando la clase **FileWriter**:
 - "Intento de acceso fallido"
 - "Usuario autenticado correctamente"
 - Cierra el archivo.
 - Abre el archivo para leer su contenido utilizando **FileReader** y **BufferedReader**.
 - Muestra cada línea leída en la consola.

Rúbrica:

0	No se cumple ninguno de los requisitos mínimos.
3	Presenta la tarea, pero el código no es correcto.
5	Se presenta la tarea y el código es correcto, pero no se demuestra su ejecución con evidencias (capturas o logs de consola).
7	Se presenta la tarea, el código es correcto, y se demuestra el funcionamiento mediante capturas o salida de consola; sin embargo, no se describen los principales métodos y clases del programa.



9	Se completa la tarea correctamente en todos sus apartados, con código funcional, capturas de consola y breve descripción de métodos y clases.
10	Se añade alguna funcionalidad extra o detalle innovador que mejore la aplicación (por ejemplo, añadir un timestamp a cada línea de log, o permitir escribir más mensajes desde teclado).

Entrega:

1. Incluye aquí el código fuente de la clase desarrollada:

```
import java.io.*;

public class LogActividad3 {
    static void main (String[]args) throws IOException {
        // Creamos un objeto con el que se crea el archivo a escribir
        FileWriter seguridad = new FileWriter("seguridad_actividad3.log");

        // Insertamos el contenido dentro del archivo se seguridad junto con su separador de linea
        seguridad.write("Intento de acceso fallido" + System.lineSeparator());
        seguridad.write("Usuario autenticado correctamente");

        seguridad.close();

        // Con FileReader leemos el archivo creado anteriormente y con el Buffer lo almacenamos en la
        memoria
        FileReader leer = new FileReader("seguridad_actividad3.log");
        BufferedReader buffer = new BufferedReader(leer);

        String texto;

        // Creamos un bucle con el que nos muestre las lineas por pantalla
        while ((texto = buffer.readLine()) != null) {
            System.out.println(texto);
        }
        // Cerramos el Buffer
        buffer.close();
    }
}
```

2. Incluye aquí captura/s de pantalla que demuestren la correcta ejecución del código (Y DONDE SEA VISIBLE EL AUTOR DE LA ACTIVIDAD):

```

1  import java.io.*;
2
3  public class LogActividad3 {
4      static void main (String[] args) throws IOException {
5          // Creamos un objeto con el que se crea el archivo a escribir
6          FileWriter seguridad = new FileWriter (@"seguridad_actividad3.log");
7
8          // Insertamos el contenido dentro del archivo de seguridad junto con su separador de línea
9          seguridad.write(@"Intento de acceso fallido" + System.lineSeparator());
10         seguridad.write(@"Usuario autenticado correctamente");
11
12         seguridad.close();
13
14         // Con FileReader leemos el archivo creado anteriormente y con el Buffer lo almacenamos en la memoria
15         FileReader leer = new FileReader (@"seguridad_actividad3.log");
16         BufferedReader buffer = new BufferedReader (leer);
17
18         String texto;
19
20         // Creamos un bucle con el que nos muestra las líneas por pantalla
21         while ((texto = buffer.readLine()) != null) {
22             System.out.println(texto);
23         }
24         // Cerramos el Buffer
25         buffer.close();
26     }
27 }
    
```

Run LogActividad3

```

C:\Users\DAW2\jdk\temurin-25.0.4.1\bin\java.exe -javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2020.2.2\lib\idea_rt.jar=35444 -Dfile.encoding=UTF-8
Intento de acceso fallido
Usuario autenticado correctamente
Process finished with exit code 0
    
```

3. Describe aquí las clases, métodos y variables principales utilizadas en el programa:

Clases:

FileWriter: Abre y escribe texto directamente en el archivo de disco (seguridad_actividad3.log).

FileReader: Abre la conexión para leer los caracteres del archivo creado.

BufferedReader: Memoria intermedia que envuelve la lectura para hacerla eficiente y leer línea por línea.

Métodos:

Write(): Escribe las cadenas de texto dentro del fichero.

ReadLine(): Lee una línea completa de texto hasta el salto de línea o devuelve null si termina el archivo.

Close(): Cierra los flujos de datos y libera los recursos del sistema.

Variables:

Seguridad (FileWriter): Objeto que controla la escritura del archivo de log.

Buffer (BufferedReader): Objeto que gestiona la lectura optimizada en memoria.

Texto:

String: Guarda temporalmente cada línea leída antes de mostrarla por consola.

